

AI & ML Lab – Week 1 Assignment

Numerical Computing and Reproducible Notebooks

1 Theme of the Week

Numerical Computing + Reproducible Notebooks

Modern AI systems rely heavily on numerical computation. This lab introduces students to efficient numerical operations using NumPy and emphasizes reproducibility — a critical requirement in scientific computing and machine learning research.

All tasks in this assignment must be implemented and executed using **Google Colab**.

2 Task 1: Vectorization Drill (2 Marks)

2.1 Objective

To understand how vectorized NumPy operations replace explicit loops, resulting in faster, cleaner, and more reliable numerical computations.

2.2 Concepts Covered

- Dot product
- Cosine similarity
- Pairwise Euclidean distance
- Loop-free vectorized implementation

2.3 Illustrative Python Examples

2.3.1 Dot Product

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

dot_product = np.dot(a, b)
print(dot_product)
```

2.3.2 Cosine Similarity

```
def cosine_similarity(x, y):
    return np.dot(x, y) / (np.linalg.norm(x) * np.linalg.norm(y))

cos_sim = cosine_similarity(a, b)
print(cos_sim)
```

2.3.3 Pairwise Distance Matrix

```
X = np.array([[1, 2], [3, 4], [5, 6]])

pairwise_dist = np.sqrt(
    np.sum((X[:, None, :] - X[None, :, :])**2, axis=-1)
)

print(pairwise_dist)
```

2.4 Deliverables

- Rollnumber-w1-task1.ipynb

Story-Based Assignment Question (Task 1)

The Recommendation Engine Challenge

A startup is building a recommendation engine where each user and product is represented as a numerical vector. The system must rapidly compute similarities between thousands of users and products in real time.

1. Explain why using Python loops for similarity computation would be inefficient.
2. Implement dot product and cosine similarity using NumPy *without loops*.
3. Demonstrate pairwise distance computation between multiple users.
4. Justify how vectorization improves performance and readability.

—

3 Task 2: Mini Data Generator (4 Marks)

3.1 Objective

To generate synthetic datasets for machine learning experiments and visualize linear separability concepts.

3.2 Concepts Covered

- Synthetic data generation
- Linearly separable vs non-separable data
- Data visualization using Matplotlib

3.3 Illustrative Python Examples

3.3.1 Linearly Separable Data

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

class1 = np.random.randn(100, 2) + [2, 2]
```

```
class2 = np.random.randn(100, 2) + [-2, -2]

plt.scatter(class1[:,0], class1[:,1], label='Class_1')
plt.scatter(class2[:,0], class2[:,1], label='Class_2')
plt.legend()
plt.show()
```

3.3.2 Non-Linearly Separable Data

```
theta = np.linspace(0, 2*np.pi, 100)
circle1 = np.c_[np.cos(theta), np.sin(theta)]
circle2 = 2 * circle1

plt.scatter(circle1[:,0], circle1[:,1])
plt.scatter(circle2[:,0], circle2[:,1])
plt.show()
```

3.4 Deliverables

- Rollnumber-w1-task2.ipynb
- Rollnumber-w1-data-plot.png

Story-Based Assignment Question (Task 2)

The Algorithm Testing Lab

A data scientist wants to test a new classification algorithm but does not have access to real data.

1. Generate a synthetic dataset with two linearly separable classes.
2. Generate another dataset where the classes are not linearly separable.
3. Plot both datasets and explain the geometric difference.
4. Discuss which types of machine learning models would perform well on each dataset.

—

4 Task 3: Reproducibility Checklist (4 Marks)

4.1 Objective

To ensure experimental results can be consistently reproduced, a fundamental requirement in AI research and industry.

4.2 Best Practices Illustrated

4.2.1 Setting Random Seeds

```
import numpy as np
np.random.seed(123)
```

4.2.2 Function-Based Structure

```
def generate_data(seed=123):  
    np.random.seed(seed)  
    return np.random.randn(10)
```

4.2.3 Saving Outputs

```
np.savetxt("output.txt", generate_data())
```

4.3 Deliverables

- Rollnumber-w1-task3.ipynb
- Roll Number-w1-notes.txt

Story-Based Assignment Question (Task 3)

The Rejected Research Paper

A research paper was rejected because reviewers could not reproduce the experimental results.

1. Identify the possible reasons why results may change between runs.
2. Modify the experiment to ensure reproducibility.
3. Clearly document assumptions and implementation choices in markdown.
4. Save outputs and explain how another researcher can verify them.

—

5 Submission Instructions

All notebooks must be executed in **Google Colab** and submitted as per the naming convention specified. Incomplete or non-reproducible submissions will be penalized.